Step-1: Loading data:

SQL — OK -- 0 rows -- 22.9ms — create raw table — Run — Prettify SQL — Wiki

```
CREATE TABLE IF NOT EXISTS superstore_raw (
  row_id INT PRIMARY KEY,
  order_id VARCHAR(50),
  order_date VARCHAR(50),
  ship_date VARCHAR(50),
  ship_mode VARCHAR(50),
  customer_id VARCHAR(50),
  customer_name VARCHAR(100),
  segment VARCHAR(50),
  country VARCHAR(100),
  city VARCHAR(100),
  state VARCHAR(100),
  postal_code VARCHAR(20),
  region VARCHAR(50),
  product_id VARCHAR(50),
  category VARCHAR(100),
  sub_category VARCHAR(100),
  product_name VARCHAR(255),
  sales NUMERIC,
  quantity INT,
  discount NUMERIC,
  profit NUMERIC
);
```

CREATE TABLE IF NOT EXISTS superstore_raw (row_id INT PRIMARY KEY, order_id VARCHAR(50), order_date VARCHAR(50), ship_date VARCHAR(50), ship_mode VARCHAR(50), customer_id VARCHAR(50), customer_name VARCHAR(100), segment VARCHAR(50), country VARCHAR(100), city VARCHAR(100), state VARCHAR(100), postal_code VARCHAR(20), region VARCHAR(50), product_id VARCHAR(50), category VARCHAR(100), sub_category VARCHAR(100), product_name VARCHAR(255), sales NUMERIC, quantity INT, discount NUMERIC, profit NUMERIC);

0 rows affected • 22.9ms

Query executed successfully. No rows returned.

1

SQL — OK -- 0 rows -- 48.9ms — load csv — Run — Prettify SQL — Wiki

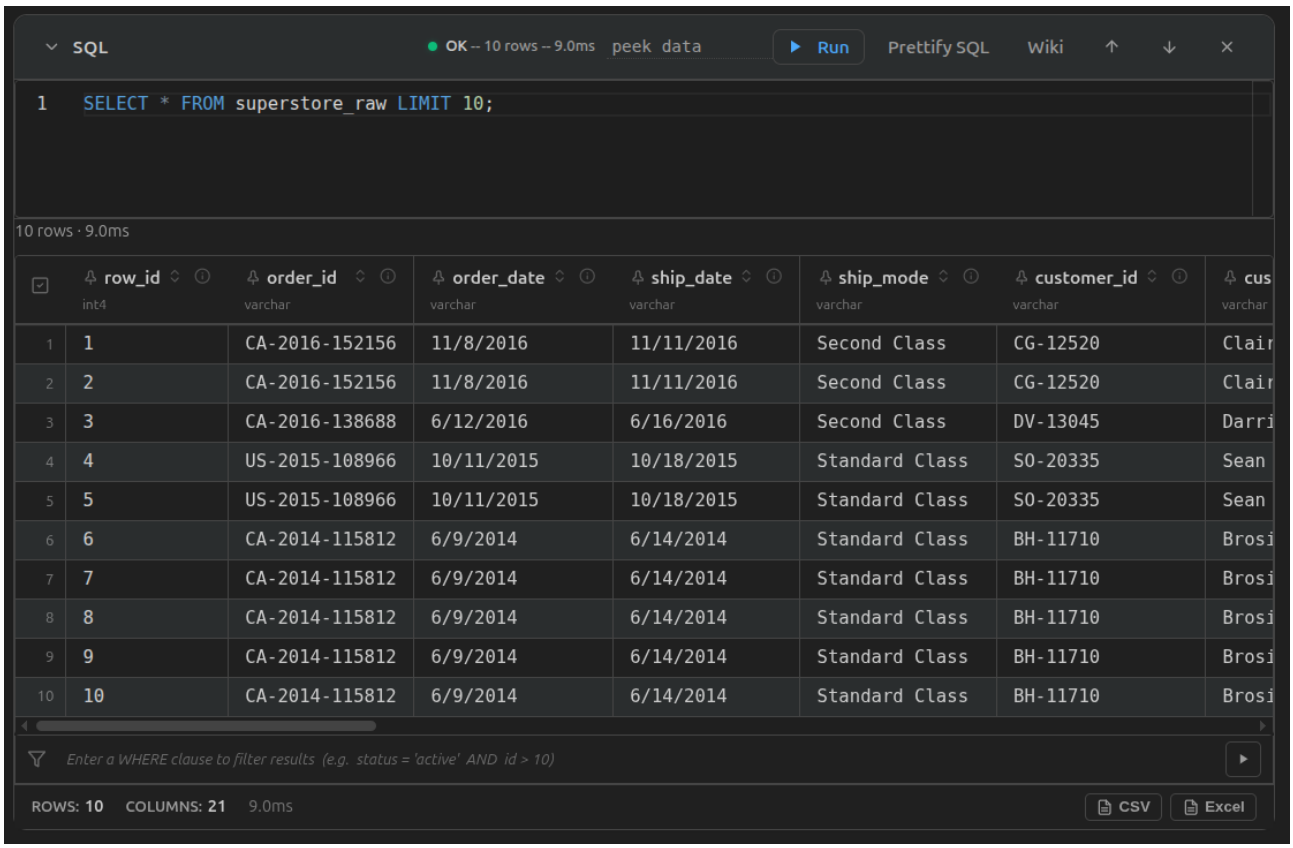
```
COPY superstore_raw
FROM '/tmp/superstore.csv'
DELIMITER ',' CSV HEADER ENCODING 'WIN1252';
```

COPY superstore_raw FROM '/tmp/superstore.csv' DELIMITER ',' CSV HEADER ENCODING 'WIN1252';

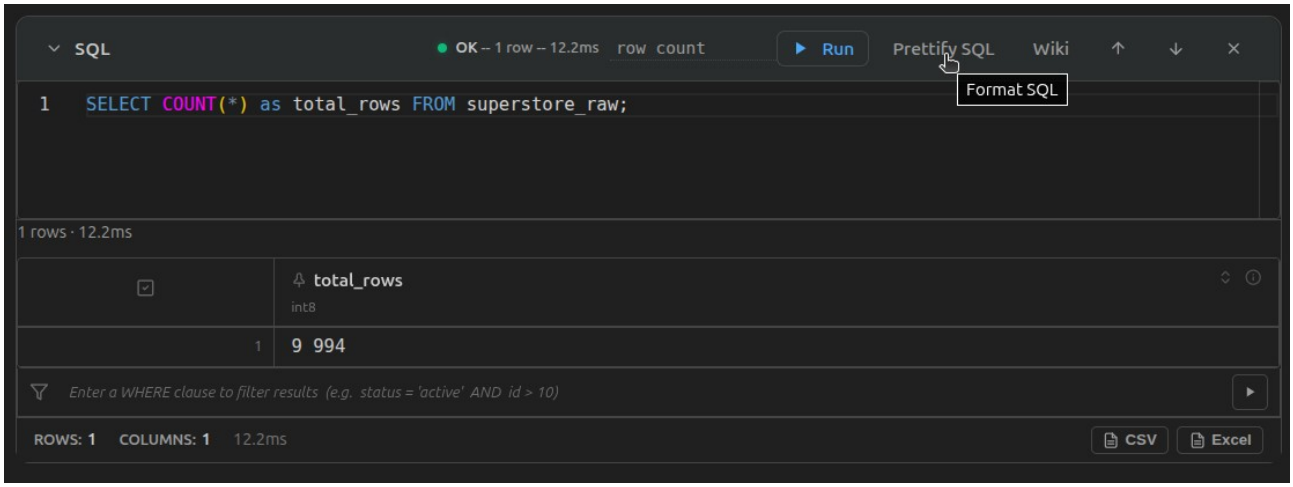
9994 rows affected • 48.9ms

Query executed successfully. No rows returned.

2

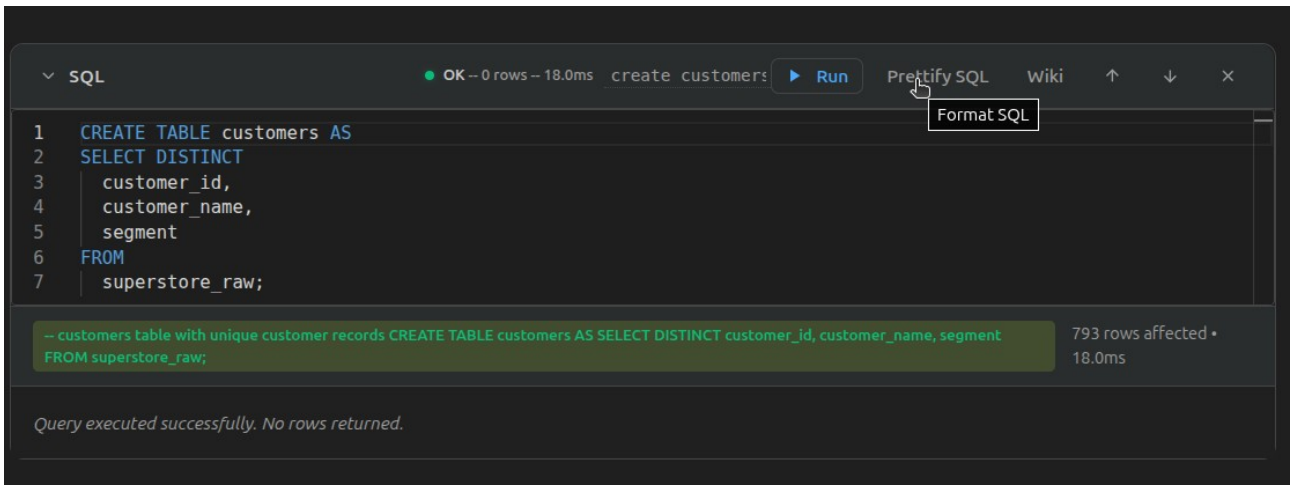


3

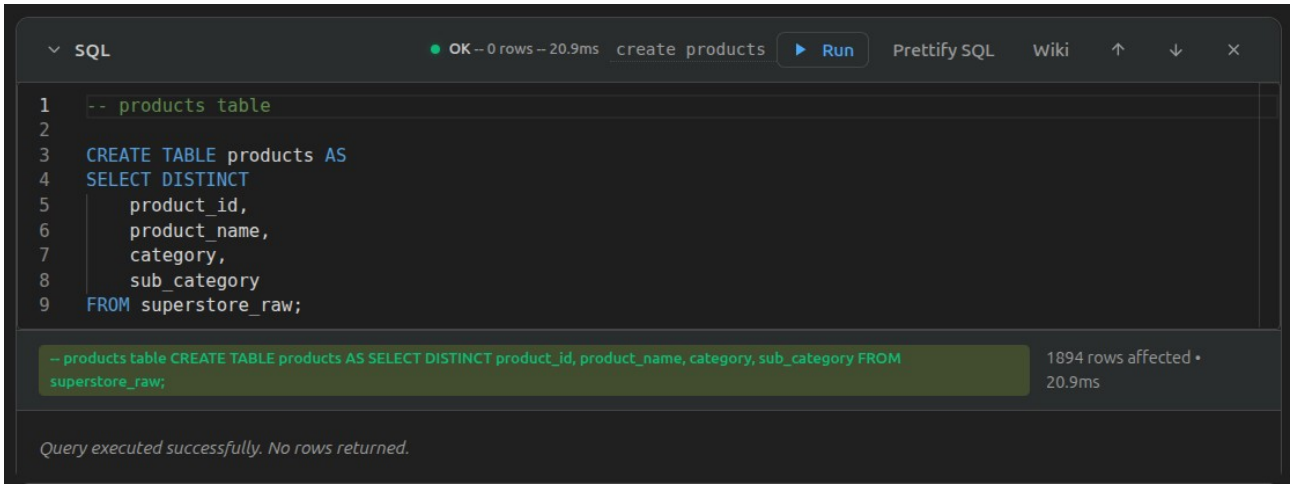


4

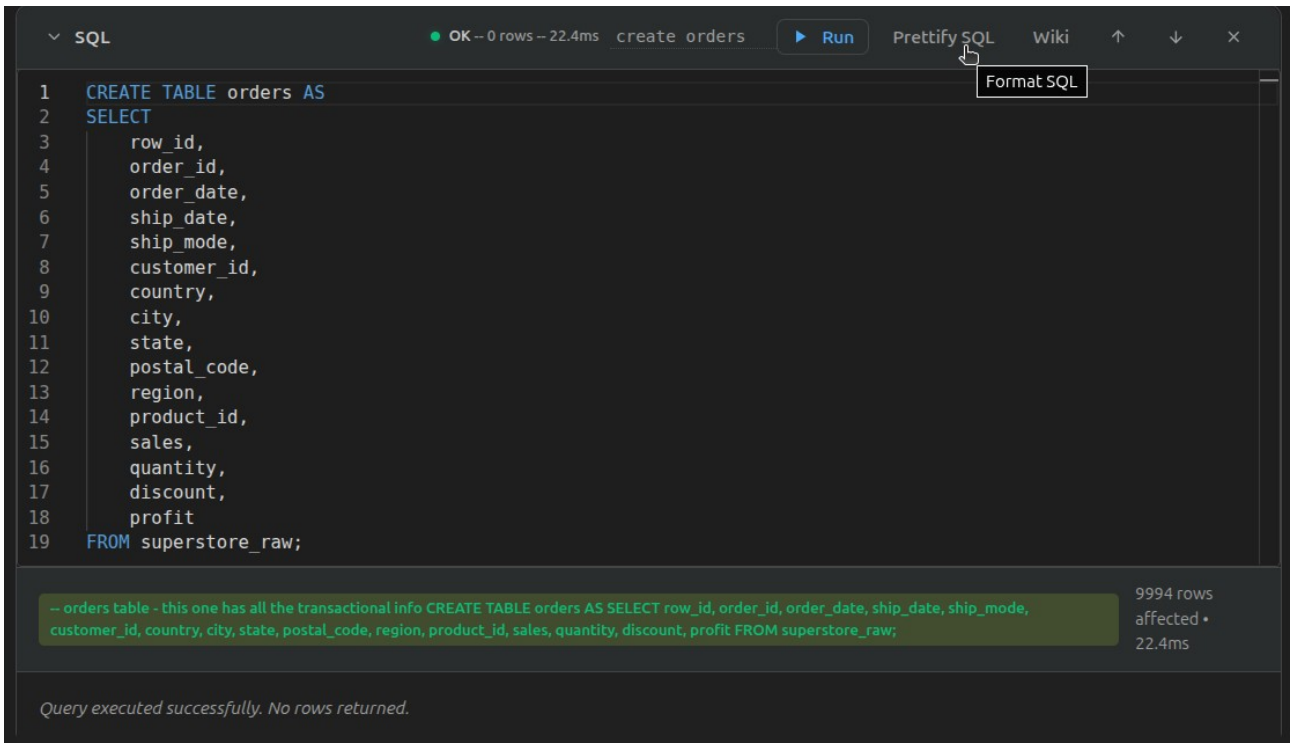
Step-2: Creating normalized tables from the raw data:


SQL
OK -- 0 rows -- 18.0ms
create customers
Run
Prettify SQL
Wiki
Format SQL
1 CREATE TABLE customers AS
2 SELECT DISTINCT
3 customer_id,
4 customer_name,
5 segment
6 FROM
7 superstore_raw;
-- customers table with unique customer records CREATE TABLE customers AS SELECT DISTINCT customer_id, customer_name, segment FROM superstore_raw;
793 rows affected • 18.0ms
Query executed successfully. No rows returned.

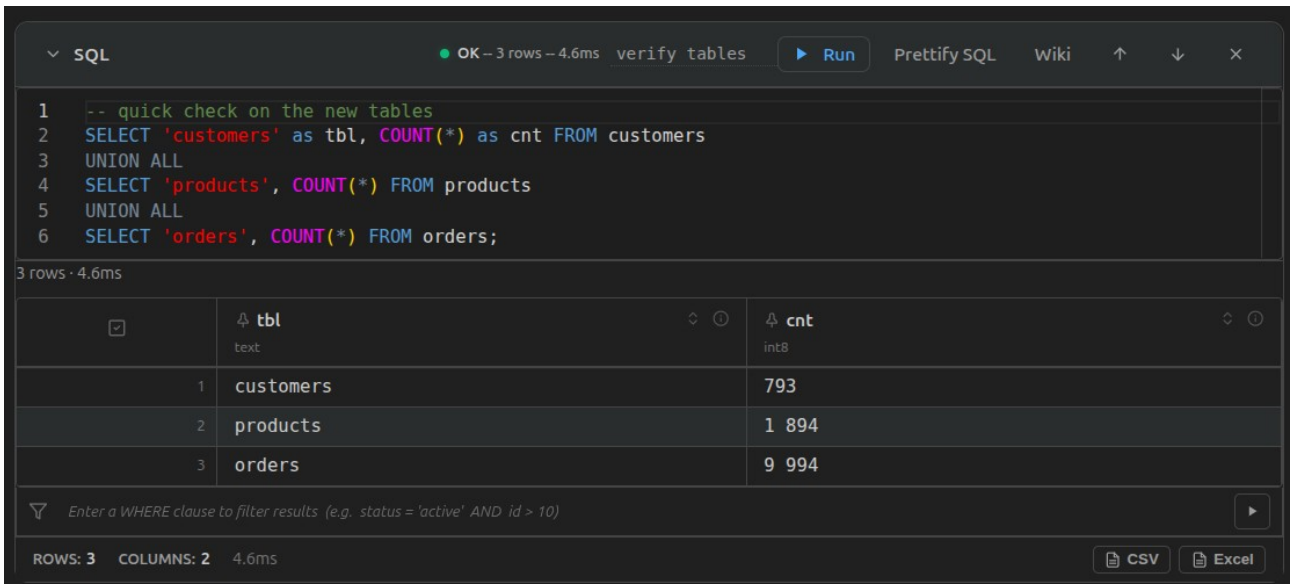
5


SQL
OK -- 0 rows -- 20.9ms
create products
Run
Prettify SQL
Wiki
1 -- products table
2
3 CREATE TABLE products AS
4 SELECT DISTINCT
5 product_id,
6 product_name,
7 category,
8 sub_category
9 FROM superstore_raw;
-- products table CREATE TABLE products AS SELECT DISTINCT product_id, product_name, category, sub_category FROM superstore_raw;
1894 rows affected • 20.9ms
Query executed successfully. No rows returned.

6

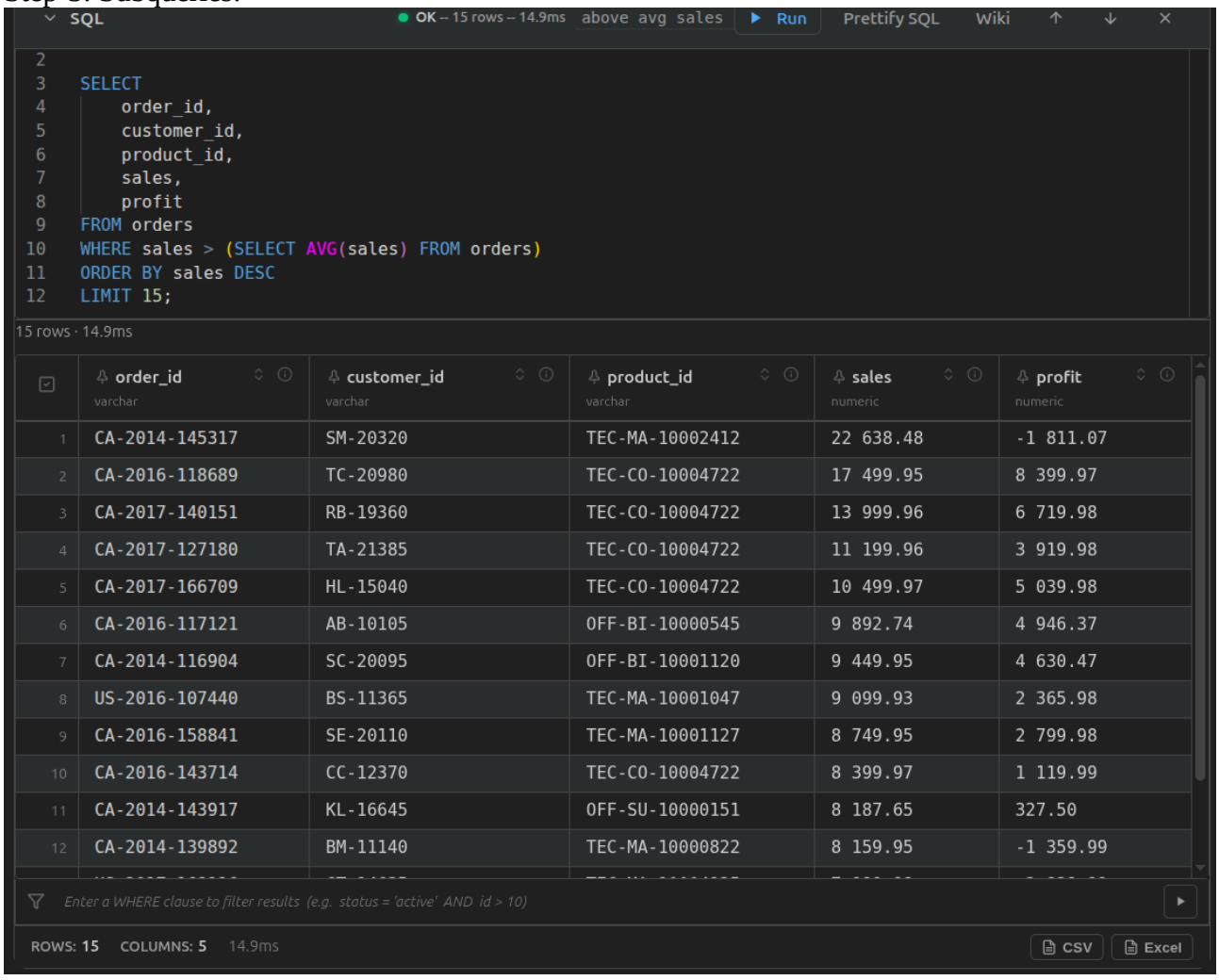

SQL
OK -- 0 rows -- 22.4ms
create orders
Run
Prettify SQL
Wiki
Format SQL
1 CREATE TABLE orders AS
2 SELECT
3 row_id,
4 order_id,
5 order_date,
6 ship_date,
7 ship_mode,
8 customer_id,
9 country,
10 city,
11 state,
12 postal_code,
13 region,
14 product_id,
15 sales,
16 quantity,
17 discount,
18 profit
19 FROM superstore_raw;
-- orders table - this one has all the transactional info CREATE TABLE orders AS SELECT row_id, order_id, order_date, ship_date, ship_mode, customer_id, country, city, state, postal_code, region, product_id, sales, quantity, discount, profit FROM superstore_raw;
9994 rows affected • 22.4ms
Query executed successfully. No rows returned.

7



8

Step-3: Subqueries:



9

SQL — OK -- 1 row -- 7.8ms — highest sale — Run — Prettify SQL — Wiki

```sql
--the single highest sales order

SELECT
    o.order_id,
    c.customer_name,
    p.product_name,
    o.sales,
    o.profit
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN products p ON o.product_id = p.product_id
WHERE o.sales = (SELECT MAX(sales) FROM orders);
```

1 rows · 7.8ms

	order_id varchar	customer_name varchar	product_name varchar	sales numeric
1	CA-2014-145317	Sean Miller	Cisco TelePresence System EX90 Videoconferencing Unit	22 638.48

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

10

```sql
SELECT
  o.order_id,
  p.category,
  p.product_name,
  o.sales
FROM
  orders o
  JOIN products p ON o.product_id = p.product_id
WHERE
  o.sales > (
    SELECT
      AVG(oo.sales)
    FROM
      orders oo
      JOIN products pp ON oo.product_id = pp.product_id
    WHERE
      pp.category = p.category
  )
ORDER BY
  o.sales DESC
LIMIT
  20;
```

20 rows · 17.55s

ler_id	category varchar	product_name varchar	sale: numeric
014-145317	Technology	Cisco TelePresence System EX90 Videoconferencing Unit	22 63
016-118689	Technology	Canon imageCLASS 2200 Advanced Copier	17 49
017-140151	Technology	Canon imageCLASS 2200 Advanced Copier	13 99
017-127180	Technology	Canon imageCLASS 2200 Advanced Copier	11 19
017-166709	Technology	Canon imageCLASS 2200 Advanced Copier	10 49
016-117121	Office Supplies	GBC Ibimaster 500 Manual ProClick Binding System	9 892
014-116904	Office Supplies	Ibico EPK-21 Electric Binding System	9 449
016-107440	Technology	3D Systems Cube Printer, 2nd Generation, Magenta	9 099
016-158841	Technology	HP Designjet T520 Inkjet Large Format Printer - 24" Color	8 749

11.orders that beat the average sales of their own category

```sql
SELECT
  customer_id,
  order_count
FROM
  (
    SELECT
      customer_id,
      COUNT(DISTINCT order_id) AS order_count
    FROM
      orders
    GROUP BY
      customer_id
  ) sub
WHERE
  order_count > (
    SELECT
      AVG(order_count)
    FROM
      (
        SELECT
          COUNT(DISTINCT order_id) AS order_count
        FROM
          orders
        GROUP BY
          customer_id
      ) avg_sub
  )
ORDER BY
  order_count DESC;
```

357 rows · 91.0ms

	customer_id varchar	order_count int8
1	EP-13915	17
2	EA-14035	13
3	PG-18820	13
4	SH-19975	13

12.customers who have placed more orders than the average customer

SQL · OK -- 5 rows -- 18.4ms · top5_products_sl · Run · Prettify SQL · Wiki

```sql
--top 5 products by total sales using subquery in FROM

SELECT
    p.product_name,
    p.category,
    prod_sales.total_sales
FROM (
    SELECT product_id, ROUND(SUM(sales),2) as total_sales
    FROM orders
    GROUP BY product_id
    ORDER BY total_sales DESC
    LIMIT 5
) prod_sales
JOIN products p ON prod_sales.product_id = p.product_id
ORDER BY prod_sales.total_sales DESC;
```

5 rows · 18.4ms

	product_name varchar	category varchar	total_sales numeric
1	Canon imageCLASS 2200 Advanced Copier	Technology	61 599.82
2	Fellowes PB500 Electric Punch Plastic Comb Binding Machine with Manual Bind	Office Supplies	27 453.38
3	Cisco TelePresence System EX90 Videoconferencing Unit	Technology	22 638.48
4	HON 5400 Series Task Chairs for Big and Tall	Furniture	21 870.58
5	GBC DocuBind TL300 Electric Binding System	Office Supplies	19 823.48

13

Step-4: CTEs:

```sql
WITH
  cust_met AS (
    SELECT
      o.customer_id,
      c.customer_name,
      c.segment,
      COUNT(DISTINCT o.order_id) AS total_orders,
      ROUND(SUM(o.sales), 2) AS total_sales,
      ROUND(SUM(o.profit), 2) AS total_profit,
      ROUND(AVG(o.sales), 2) AS avg_order_value
    FROM
      orders o
      JOIN customers c ON o.customer_id = c.customer_id
    GROUP BY
      o.customer_id,
      c.customer_name,
      c.segment
  )
SELECT
  *
FROM
  cust_met
ORDER BY
  total_sales DESC
LIMIT
  15;
```

Format SQL

15 rows · 56.0ms

stomer_id	customer_name (varchar)	segment (varchar)	total_orders (int8)	total_sales (numeric)	total_profit (numeric)	avg (numeric)
0320	Sean Miller	Home Office	5	25 043.05	-1 980.74	1 66
0980	Tamara Chand	Corporate	5	19 052.22	8 981.32	1 58
9360	Raymond Buch	Consumer	6	15 117.34	6 976.10	839.
1385	Tom Ashbrook	Home Office	4	14 595.62	4 703.79	1 45
0105	Adrian Barton	Consumer	10	14 473.57	5 444.81	723.
6645	Ken Lonsdale	Consumer	12	14 175.23	806.86	488.

14. customer-level sales summary using CTE

Step-5: Window functions:

15

```sql
SELECT
  customer_id,
  order_id,
  sales,
  ROW_NUMBER() OVER (
    PARTITION BY
      customer_id
    ORDER BY
      sales DESC
  ) AS row_num
FROM
  orders
LIMIT
  20;
```

20 rows · 23.8ms

	customer_id (varchar)	order_id (varchar)	sales (numeric)	row_num (int8)
1	AA-10315	CA-2016-103982	3 930.07	1
2	AA-10315	CA-2014-128055	673.56	2
3	AA-10315	CA-2016-103982	431.97	3
4	AA-10315	CA-2017-147039	362.94	4
5	AA-10315	CA-2014-128055	52.98	5
6	AA-10315	CA-2016-103982	41.72	6
7	AA-10315	CA-2015-121391	26.96	7
8	AA-10315	CA-2014-138100	14.94	8
9	AA-10315	CA-2014-138100	14.56	9
10	AA-10315	CA-2017-147039	11.54	10
11	AA-10315	CA-2016-103982	2.30	11
12	AA-10375	CA-2016-131065	499.98	1

```sql
SELECT * FROM (
    SELECT
        o.customer_id,
        c.customer_name,
        o.order_id,
        o.sales,
        ROW_NUMBER() OVER(
            PARTITION BY o.customer_id
            ORDER BY o.sales DESC
        ) as row_num
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
) ranked
WHERE row_num = 1
ORDER BY sales DESC
LIMIT 10;
```

10 rows · 28.6ms

	customer_id varchar	customer_name varchar	order_id varchar	sales numeric	row_num int8
1	SM-20320	Sean Miller	CA-2014-145317	22 638.48	1
2	TC-20980	Tamara Chand	CA-2016-118689	17 499.95	1
3	RB-19360	Raymond Buch	CA-2017-140151	13 999.96	1
4	TA-21385	Tom Ashbrook	CA-2017-127180	11 199.96	1
5	HL-15040	Hunter Lopez	CA-2017-166709	10 499.97	1
6	AB-10105	Adrian Barton	CA-2016-117121	9 892.74	1
7	SC-20095	Sanjit Chand	CA-2014-116904	9 449.95	1
8	BS-11365	Bill Shonely	US-2016-107440	9 099.93	1
9	SE-20110	Sanjit Engle	CA-2016-158841	8 749.95	1
10	CC-12370	Christopher Conant	CA-2016-143714	8 399.97	1

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 10 COLUMNS: 5 28.6ms CSV Excel

17.get each customer's top order (their highest sale)

```sql
WITH
  cust_totals AS (
    SELECT
      customer_id,
      ROUND(SUM(sales), 2) AS total_sales
    FROM
      orders
    GROUP BY
      customer_id
  )
SELECT
  customer_id,
  total_sales,
  RANK() OVER (
    ORDER BY
      total_sales DESC
  ) AS rank_num,
  DENSE_RANK() OVER (
    ORDER BY
      total_sales DESC
  ) AS dense_rank_num,
  ROW_NUMBER() OVER (
    ORDER BY
      total_sales DESC
  ) AS row_num
FROM
  cust_totals
LIMIT
  20;
```

20 rows · 16.2ms

	customer_id (varchar)	total_sales (numeric)	rank_num (int8)	dense_rank_num (int8)	row_num (int8)
1	SM-20320	25 043.05	1	1	1
2	TC-20980	19 052.22	2	2	2
3	RB-19360	15 117.34	3	3	3
4	TA-21385	14 595.62	4	4	4

18.rank vs dense_rank vs row_num comparison on customer total sales

```sql
WITH product_totals AS (
    SELECT
        p.product_id,
        p.product_name,
        p.category,
        ROUND(SUM(o.sales),2) as total_sales
    FROM orders o
    JOIN products p ON o.product_id = p.product_id
    GROUP BY p.product_id, p.product_name, p.category
)
SELECT
    category,
    product_name,
    total_sales,
    DENSE_RANK() OVER(
        PARTITION BY category
        ORDER BY total_sales DESC
    ) as category_rank
FROM product_totals
ORDER BY category, category_rank
LIMIT 25;
```

25 rows · 32.3ms

	total_sales numeric	category_rank int8
; Task Chairs for Big and Tall	21 870.58	1
.s Royal Lawyers Bookcase, Royale Cherry Finish	15 610.97	2
ngular Conference Table Tops	12 995.29	3
ecutive Leather Low-Back Tilter	12 975.38	4
Hills Library, Woodland Oak Finish	12 921.64	5
ecutive Suite Bookcases	12 921.64	5
ding Chair	11 572.78	6
ric Upholstered Stacking Chairs, Rounded Back	10 637.53	7

19.rank products within each category by total sales

```sql
SELECT
    o.customer_id,
    c.customer_name,
    o.order_id,
    o.order_date,
    o.sales,
    SUM(o.sales) OVER(
        PARTITION BY o.customer_id
        ORDER BY TO_DATE(o.order_date,'MM/DD/YYYY')
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) as running_total
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
ORDER BY o.customer_id, TO_DATE(o.order_date,'MM/DD/YYYY')
LIMIT 25;
```

25 rows · 22.4ms

	customer_id varchar	customer_name varchar	order_id varchar	order_date varchar	sales numeric	running_total numeric
1	AA-10315	Alex Avila	CA-2014-128055	3/31/2014	52.98	52.98
2	AA-10315	Alex Avila	CA-2014-128055	3/31/2014	673.56	726.54
3	AA-10315	Alex Avila	CA-2014-138100	9/15/2014	14.56	741.10
4	AA-10315	Alex Avila	CA-2014-138100	9/15/2014	14.94	756.04
5	AA-10315	Alex Avila	CA-2015-121391	10/4/2015	26.96	783.00
6	AA-10315	Alex Avila	CA-2016-103982	3/3/2016	3 930.07	4 713.08
7	AA-10315	Alex Avila	CA-2016-103982	3/3/2016	431.97	5 145.05
8	AA-10315	Alex Avila	CA-2016-103982	3/3/2016	41.72	5 186.77
9	AA-10315	Alex Avila	CA-2016-103982	3/3/2016	2.30	5 189.08
10	AA-10315	Alex Avila	CA-2017-147039	6/29/2017	362.94	5 552.02
11	AA-10315	Alex Avila	CA-2017-147039	6/29/2017	11.54	5 563.56
12	AA-10375	Allen Armold	CA-2014-158064	4/21/2014	16.52	16.52

20.running total of sales per customer (ordered by order date)

Step-6: Combining join, cte, windows functions:

```sql
WITH cust_agg AS (
    SELECT
        o.customer_id,
        c.customer_name,
        c.segment,
        COUNT(DISTINCT o.order_id) as num_orders,
        ROUND(SUM(o.sales), 2) as total_sales,
        ROUND(SUM(o.profit), 2) as total_profit,
        ROUND(AVG(o.sales), 2) as avg_sale
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    GROUP BY o.customer_id, c.customer_name, c.segment
)
SELECT
    customer_name,
    segment,
    num_orders,
    total_sales,
    total_profit,
    avg_sale,
    RANK() OVER(ORDER BY total_sales DESC) as overall_rank,
    RANK() OVER(PARTITION BY segment ORDER BY total_sales DESC) as segment_rank
FROM cust_agg
ORDER BY overall_rank
LIMIT 20;
```

20 rows · 38.2ms

	customer_name varchar	segment varchar	num_orders int8	total_sales numeric	total_profit numeric	avg_sale numeric
1	Sean Miller	Home Office	5	25 043.05	-1 980.74	1 669.54
2	Tamara Chand	Corporate	5	19 052.22	8 981.32	1 587.68
3	Raymond Buch	Consumer	6	15 117.34	6 976.10	839.85
4	Tom Ashbrook	Home Office	4	14 595.62	4 703.79	1 459.56
5	Adrian Barton	Consumer	10	14 473.57	5 444.81	723.68
6	Ken Lonsdale	Consumer	12	14 175.23	806.86	488.80
7	Sanjit Chand	Consumer	9	14 142.33	5 757.41	642.83
8	Hunter Lopez	Consumer	6	12 873.30	5 622.43	1 170.30
9	Sanjit Engle	Consumer	11	12 209.44	2 650.68	642.60
10	Christopher Conant	Consumer	5	12 129.07	2 177.05	1 102.64
11	Todd Sumrall	Corporate	6	11 891.75	2 371.71	792.78
12	Greg Tran	Consumer	11	11 820.12	2 163.43	407.59

21.full customer ranking report with segment-level ranks

```sql
--top 3 customers per segment

WITH cust_sales AS (
    SELECT
        o.customer_id,
        c.customer_name,
        c.segment,
        ROUND(SUM(o.sales),2) as total_sales
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    GROUP BY o.customer_id, c.customer_name, c.segment
),
ranked AS (
    SELECT
        *,
        ROW_NUMBER() OVER(PARTITION BY segment ORDER BY total_sales DESC) as seg_rank
    FROM cust_sales
)
SELECT
    segment,
    customer_name,
    total_sales,
    seg_rank
FROM ranked
WHERE seg_rank <= 3
ORDER BY segment, seg_rank;
```

9 rows · 26.4ms

	segment varchar	customer_name varchar	total_sales numeric	seg_rank int8
1	Consumer	Raymond Buch	15 117.34	1
2	Consumer	Adrian Barton	14 473.57	2
3	Consumer	Ken Lonsdale	14 175.23	3
4	Corporate	Tamara Chand	19 052.22	1
5	Corporate	Todd Sumrall	11 891.75	2
6	Corporate	Bill Shonely	10 501.65	3
7	Home Office	Sean Miller	25 043.05	1
8	Home Office	Tom Ashbrook	14 595.62	2
9	Home Office	Maria Etezadi	10 663.73	3

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 9 COLUMNS: 4 26.4ms CSV Excel

```sql
--each customer's best selling product category

WITH cust_cat_sales AS (
    SELECT
        o.customer_id,
        c.customer_name,
        p.category,
        ROUND(SUM(o.sales),2) as cat_sales
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    JOIN products p ON o.product_id = p.product_id
    GROUP BY o.customer_id, c.customer_name, p.category
),
ranked_cats AS (
    SELECT *,
        ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY cat_sales DESC) as row_num
    FROM cust_cat_sales
)
SELECT
    customer_name,
    category as top_category,
    cat_sales
FROM ranked_cats
WHERE row_num = 1
ORDER BY cat_sales DESC
LIMIT 15;
```

15 rows · 36.1ms

	customer_name varchar	top_category varchar	cat_sales numeric
1	Sean Miller	Technology	23 481.51
2	Tamara Chand	Technology	17 997.95
3	Raymond Buch	Technology	14 265.42
4	Tom Ashbrook	Technology	13 709.96
5	Sanjit Chand	Office Supplies	12 081.26
6	Hunter Lopez	Technology	11 640.81
7	Adrian Barton	Office Supplies	11 489.26
8	Christopher Conant	Technology	11 322.32
9	Greg Tran	Furniture	10 227.10
10	Ken Lonsdale	Office Supplies	9 708.67
11	Sanjit Engle	Technology	9 351.99
12	Bill Shonely	Technology	9 106.83

Step-7: some business queries:

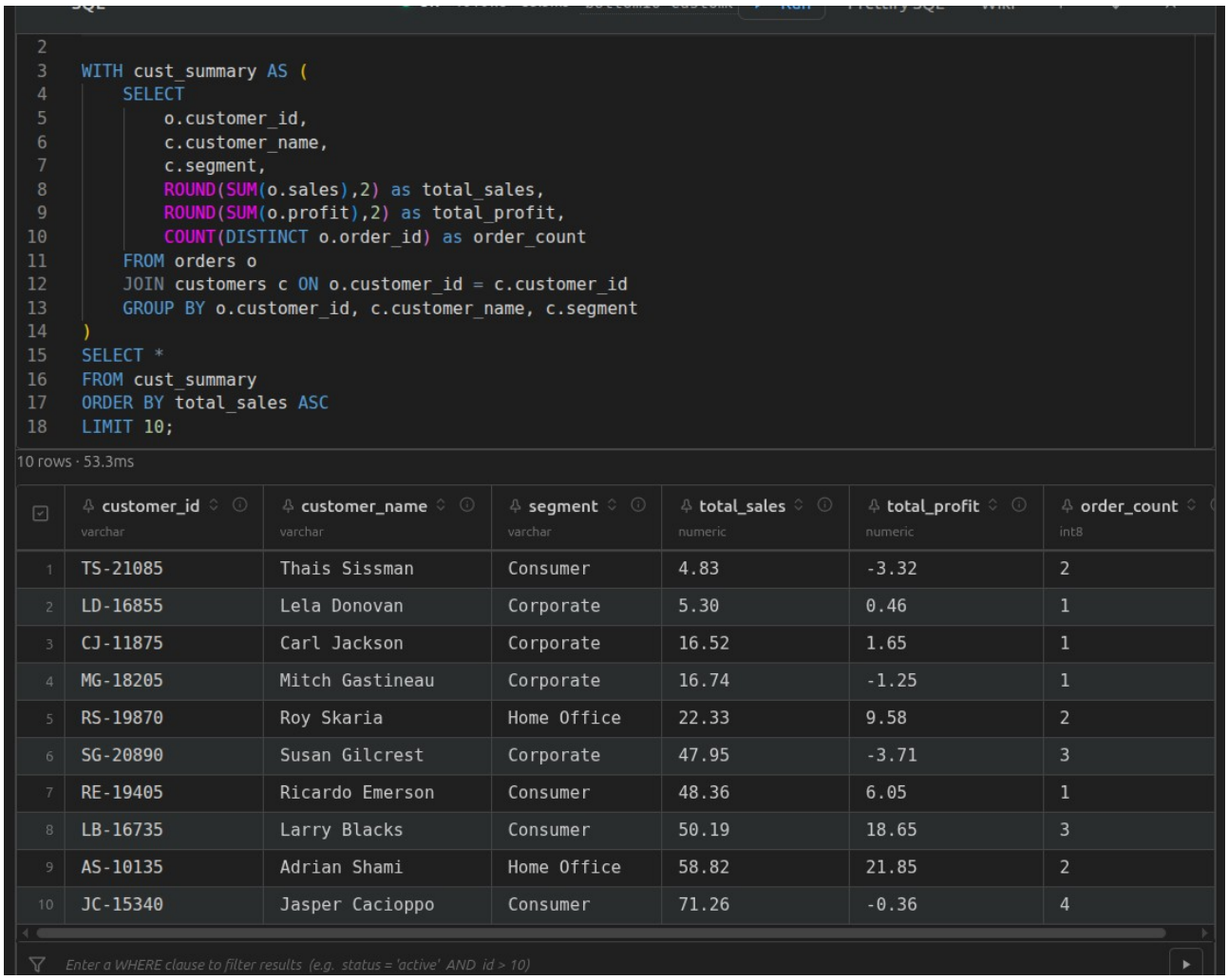
```sql
WITH cust_summary AS (
    SELECT
        o.customer_id,
        c.customer_name,
        c.segment,
        ROUND(SUM(o.sales),2) as total_sales,
        ROUND(SUM(o.profit),2) as total_profit,
        COUNT(DISTINCT o.order_id) as order_count
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    GROUP BY o.customer_id, c.customer_name, c.segment
)
SELECT *
FROM cust_summary
ORDER BY total_sales DESC
LIMIT 10;
```

10 rows · 34.3ms

	customer_id varchar	customer_name varchar	segment varchar	total_sales numeric	total_profit numeric	order_count int8
1	SM-20320	Sean Miller	Home Office	25 043.05	-1 980.74	5
2	TC-20980	Tamara Chand	Corporate	19 052.22	8 981.32	5
3	RB-19360	Raymond Buch	Consumer	15 117.34	6 976.10	6
4	TA-21385	Tom Ashbrook	Home Office	14 595.62	4 703.79	4
5	AB-10105	Adrian Barton	Consumer	14 473.57	5 444.81	10
6	KL-16645	Ken Lonsdale	Consumer	14 175.23	806.86	12
7	SC-20095	Sanjit Chand	Consumer	14 142.33	5 757.41	9
8	HL-15040	Hunter Lopez	Consumer	12 873.30	5 622.43	6
9	SE-20110	Sanjit Engle	Consumer	12 209.44	2 650.68	11
10	CC-12370	Christopher Conant	Consumer	12 129.07	2 177.05	5

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

24. top 10 customers by total sales



```sql
WITH cust_summary AS (
    SELECT
        o.customer_id,
        c.customer_name,
        c.segment,
        ROUND(SUM(o.sales),2) as total_sales,
        ROUND(SUM(o.profit),2) as total_profit,
        COUNT(DISTINCT o.order_id) as order_count
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    GROUP BY o.customer_id, c.customer_name, c.segment
)
SELECT *
FROM cust_summary
ORDER BY total_sales ASC
LIMIT 10;
```

10 rows · 53.3ms

	customer_id varchar	customer_name varchar	segment varchar	total_sales numeric	total_profit numeric	order_count int8
1	TS-21085	Thais Sissman	Consumer	4.83	-3.32	2
2	LD-16855	Lela Donovan	Corporate	5.30	0.46	1
3	CJ-11875	Carl Jackson	Corporate	16.52	1.65	1
4	MG-18205	Mitch Gastineau	Corporate	16.74	-1.25	1
5	RS-19870	Roy Skaria	Home Office	22.33	9.58	2
6	SG-20890	Susan Gilcrest	Corporate	47.95	-3.71	3
7	RE-19405	Ricardo Emerson	Consumer	48.36	6.05	1
8	LB-16735	Larry Blacks	Consumer	50.19	18.65	3
9	AS-10135	Adrian Shami	Home Office	58.82	21.85	2
10	JC-15340	Jasper Cacioppo	Consumer	71.26	-0.36	4

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

25. bottom 10 customers

```sql
WITH cust_orders AS (
    SELECT
        o.customer_id,
        c.customer_name,
        c.segment,
        COUNT(DISTINCT o.order_id) as order_count,
        ROUND(SUM(o.sales),2) as total_sales
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    GROUP BY o.customer_id, c.customer_name, c.segment
)
SELECT *
FROM cust_orders
WHERE order_count = 1
ORDER BY total_sales DESC;
```

12 rows · 33.3ms

	customer_id varchar	customer_name varchar	segment varchar	order_count int8	total_sales numeric
1	JC-15385	Jenna Caffey	Consumer	1	1 058.11
2	SM-20905	Susan MacKendrick	Consumer	1	1 043.04
3	TC-21145	Theresa Coyne	Corporate	1	1 038.26
4	JR-15700	Jocasta Rupert	Consumer	1	863.88
5	PH-18790	Patricia Hirasaki	Home Office	1	729.65
6	AO-10810	Anthony O'Donnell	Corporate	1	161.28
7	RM-19750	Roland Murray	Consumer	1	98.35
8	AR-10570	Anemone Ratner	Consumer	1	88.15
9	RE-19405	Ricardo Emerson	Consumer	1	48.36
10	MG-18205	Mitch Gastineau	Corporate	1	16.74
11	CJ-11875	Carl Jackson	Corporate	1	16.52
12	LD-16855	Lela Donovan	Corporate	1	5.30

26. customers who payed for single order
26.customers who placed a single order

SQL

OK – 2 rows – 17.6ms customer_distrit Run Prettify SQL Wiki

```sql
--customer distribution summary,i.e., how many are above avg vs below avg
WITH
  cust_totals AS (
    SELECT
      customer_id,
      SUM(sales) AS total_sales
    FROM
      orders
    GROUP BY
      customer_id
  ),
  avg_calc AS (
    SELECT
      AVG(total_sales) AS avg_sales
    FROM
      cust_totals
  )
SELECT
  CASE
    WHEN ct.total_sales > ac.avg_sales THEN 'Above Average'
    ELSE 'Below Average'
  END AS cust_grp,
  COUNT(*) AS num_customers,
  ROUND(AVG(ct.total_sales), 2) AS group_avg_sales,
  ROUND(MIN(ct.total_sales), 2) AS min_sales,
  ROUND(MAX(ct.total_sales), 2) AS max_sales
FROM
  cust_totals ct
  CROSS JOIN avg_calc ac
GROUP BY
  cust_grp
ORDER BY
  group_avg_sales DESC;
```

2 rows · 17.6ms

	cust_grp (text)	num_customers (int8)	group_avg_sales (numeric)	min_sales (numeric)	max_sales (numeric)
1	Above Average	294	5 362.37	2 900.03	25 043.05
2	Below Average	499	1 444.22	4.83	2 893.46

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 2 COLUMNS: 5 17.6ms CSV Excel